



```
In [34]: import numpy as np
import matplotlib.pyplot as plt
```

```
In [35]: np.random.seed(42)
X = np.random.rand(100, 1) - 0.5
y = 3*X[:, 0]**2 + 0.05 * np.random.randn(100)
```

```
In [36]: import pandas as pd
```

```
In [37]: df = pd.DataFrame()
```

```
In [38]: df['X'] = X.reshape(100)
df['y'] = y
```

```
In [39]: df
```

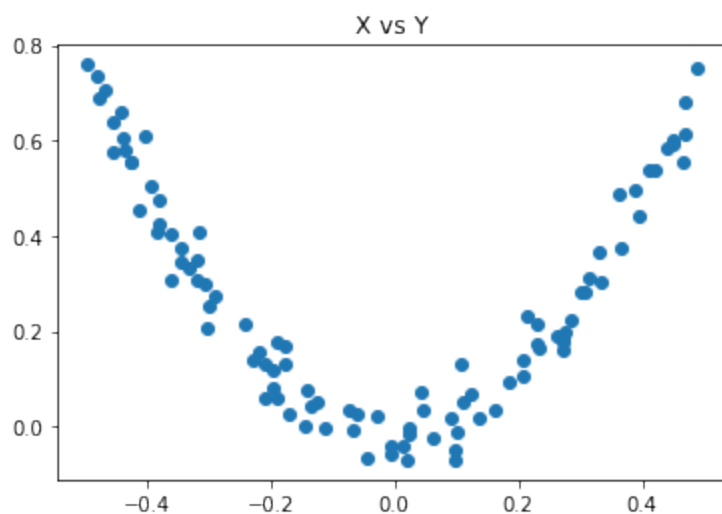
```
Out[39]:
```

	X	y
0	-0.125460	0.051573
1	0.450714	0.594480
2	0.231994	0.166052
3	0.098658	-0.070178
4	-0.343981	0.343986
...	...	...
95	-0.006204	-0.040675
96	0.022733	-0.002305
97	-0.072459	0.032809
98	-0.474581	0.689516
99	-0.392109	0.502607

100 rows × 2 columns

```
In [40]: plt.scatter(df['X'],df['y'])
plt.title('X vs Y')
```

```
Out[40]: Text(0.5, 1.0, 'X vs Y')
```



```
In [41]: df['pred1'] = df['y'].mean()
```

```
In [42]: df
```

```
Out[42]:
```

	X	y	pred1
0	-0.125460	0.051573	0.265458
1	0.450714	0.594480	0.265458
2	0.231994	0.166052	0.265458
3	0.098658	-0.070178	0.265458
4	-0.343981	0.343986	0.265458
...	...	...	...
95	-0.006204	-0.040675	0.265458
96	0.022733	-0.002305	0.265458
97	-0.072459	0.032809	0.265458
98	-0.474581	0.689516	0.265458
99	-0.392109	0.502607	0.265458

100 rows × 3 columns

```
In [43]: df['res1'] = df['y'] - df['pred1']
```

```
In [44]: df
```

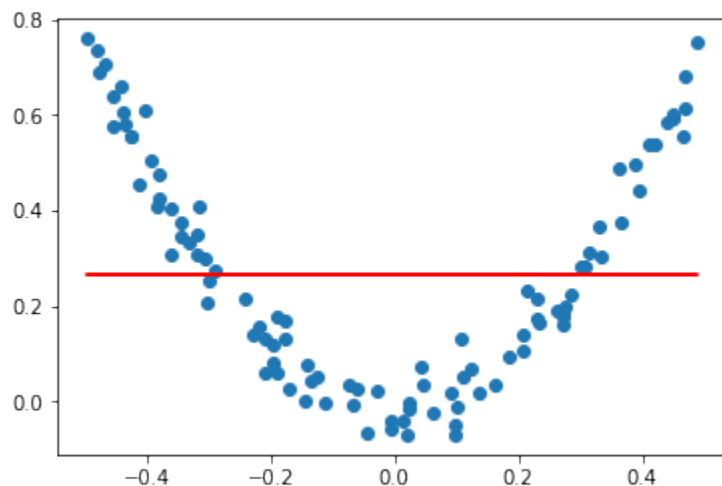
Out[44]:

	X	y	pred1	res1
0	-0.125460	0.051573	0.265458	-0.213885
1	0.450714	0.594480	0.265458	0.329021
2	0.231994	0.166052	0.265458	-0.099407
3	0.098658	-0.070178	0.265458	-0.335636
4	-0.343981	0.343986	0.265458	0.078528
...	...	...	...	...
95	-0.006204	-0.040675	0.265458	-0.306133
96	0.022733	-0.002305	0.265458	-0.267763
97	-0.072459	0.032809	0.265458	-0.232650
98	-0.474581	0.689516	0.265458	0.424057
99	-0.392109	0.502607	0.265458	0.237148

100 rows × 4 columns

```
In [45]: plt.scatter(df['X'],df['y'])
plt.plot(df['X'],df['pred1'],color='red')
```

Out[45]: [



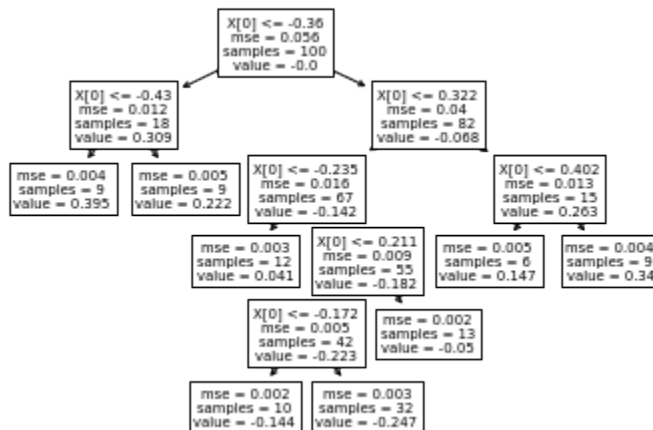
```
In [46]: from sklearn.tree import DecisionTreeRegressor
```

```
In [17]: tree1 = DecisionTreeRegressor(max_leaf_nodes=8)
```

```
In [47]: tree1.fit(df['X'].values.reshape(100,1),df['res1'].values)
```

```
Out[47]: DecisionTreeRegressor(ccp_alpha=0.0, criterion='mse', max_depth=None,
                                max_features=None, max_leaf_nodes=8,
                                min_impurity_decrease=0.0, min_impurity_split=None,
                                min_samples_leaf=1, min_samples_split=2,
                                min_weight_fraction_leaf=0.0, presort='deprecated',
                                random_state=None, splitter='best')
```

```
In [48]: from sklearn.tree import plot_tree
plot_tree(tree1)
plt.show()
```

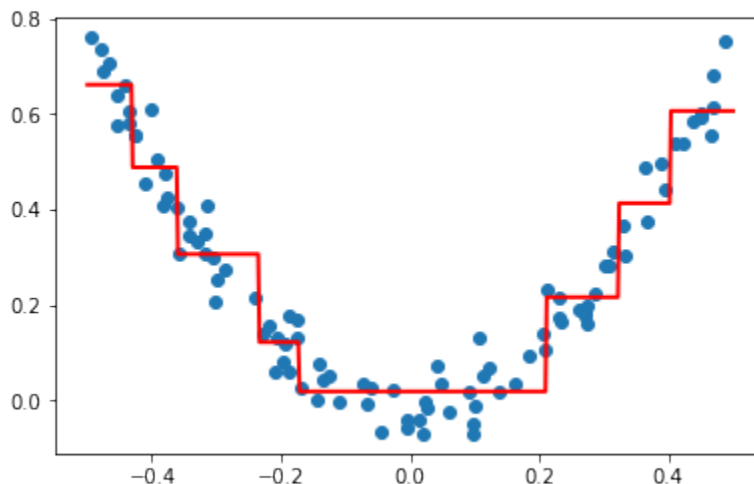


```
In [49]: # generating X_test
X_test = np.linspace(-0.5, 0.5, 500)
```

```
In [50]: y_pred = 0.265458 + tree1.predict(X_test.reshape(500, 1))
```

```
In [51]: plt.figure(figsize=(14,4))
plt.subplot(121)
plt.plot(X_test, y_pred, linewidth=2,color='red')
plt.scatter(df['X'],df['y'])
```

```
Out[51]: <matplotlib.collections.PathCollection at 0x7f40df5ee810>
```



```
In [52]: df['pred2'] = 0.265458 + tree1.predict(df['X'].values.reshape(100,1))
```

```
In [53]: df
```

```
Out[53]:
```

	X	y	pred1	res1	pred2
0	-0.125460	0.051573	0.265458	-0.213885	0.018319
1	0.450714	0.594480	0.265458	0.329021	0.605884
2	0.231994	0.166052	0.265458	-0.099407	0.215784
3	0.098658	-0.070178	0.265458	-0.335636	0.018319
4	-0.343981	0.343986	0.265458	0.078528	0.305964
...	...	...	...	...	...
95	-0.006204	-0.040675	0.265458	-0.306133	0.018319
96	0.022733	-0.002305	0.265458	-0.267763	0.018319
97	-0.072459	0.032809	0.265458	-0.232650	0.018319
98	-0.474581	0.689516	0.265458	0.424057	0.660912
99	-0.392109	0.502607	0.265458	0.237148	0.487796

100 rows × 5 columns

```
In [54]: df['res2'] = df['y'] - df['pred2']
```

```
In [55]: df
```

Out[55]:

	X	y	pred1	res1	pred2	res2
0	-0.125460	0.051573	0.265458	-0.213885	0.018319	0.033254
1	0.450714	0.594480	0.265458	0.329021	0.605884	-0.011404
2	0.231994	0.166052	0.265458	-0.099407	0.215784	-0.049732
3	0.098658	-0.070178	0.265458	-0.335636	0.018319	-0.088497
4	-0.343981	0.343986	0.265458	0.078528	0.305964	0.038022
...	...	...	...	...	...	...
95	-0.006204	-0.040675	0.265458	-0.306133	0.018319	-0.058994
96	0.022733	-0.002305	0.265458	-0.267763	0.018319	-0.020624
97	-0.072459	0.032809	0.265458	-0.232650	0.018319	0.014489
98	-0.474581	0.689516	0.265458	0.424057	0.660912	0.028604
99	-0.392109	0.502607	0.265458	0.237148	0.487796	0.014810

100 rows × 6 columns

```
In [56]: tree2 = DecisionTreeRegressor(max_leaf_nodes=8)
```

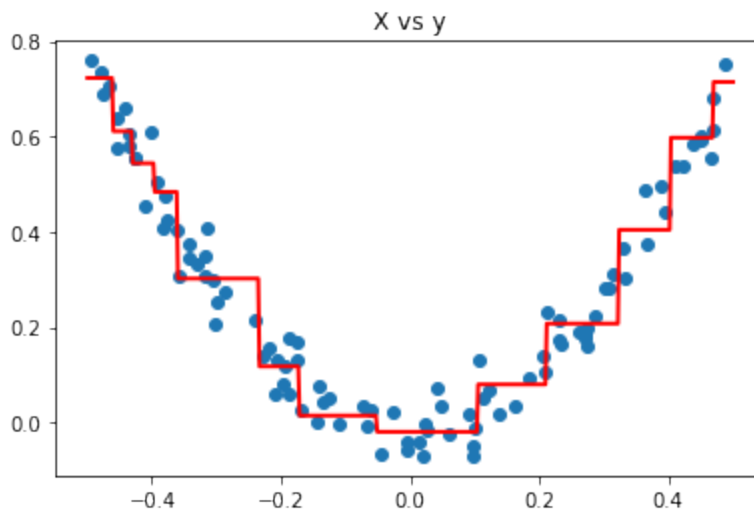
```
In [57]: tree2.fit(df['X'].values.reshape(100,1),df['res2'].values)
```

```
Out[57]: DecisionTreeRegressor(ccp_alpha=0.0, criterion='mse', max_depth=None,
                                max_features=None, max_leaf_nodes=8,
                                min_impurity_decrease=0.0, min_impurity_split=None,
                                min_samples_leaf=1, min_samples_split=2,
                                min_weight_fraction_leaf=0.0, presort='deprecated',
                                random_state=None, splitter='best')
```

```
In [58]: y_pred = 0.265458 + sum(regressor.predict(X_test.reshape(-1, 1)) for regressor
```

```
In [59]: plt.figure(figsize=(14,4))
plt.subplot(121)
plt.plot(X_test, y_pred, linewidth=2,color='red')
plt.scatter(df['X'],df['y'])
plt.title('X vs y')
```

```
Out[59]: Text(0.5, 1.0, 'X vs y')
```



```
In [60]: def gradient_boost(X,y,number,lr,count=1,regs=[],foo=None):

    if number == 0:
        return
    else:
        # do gradient boosting

        if count > 1:
            y = y - regs[-1].predict(X)
        else:
            foo = y

        tree_reg = DecisionTreeRegressor(max_depth=5, random_state=42)
        tree_reg.fit(X, y)

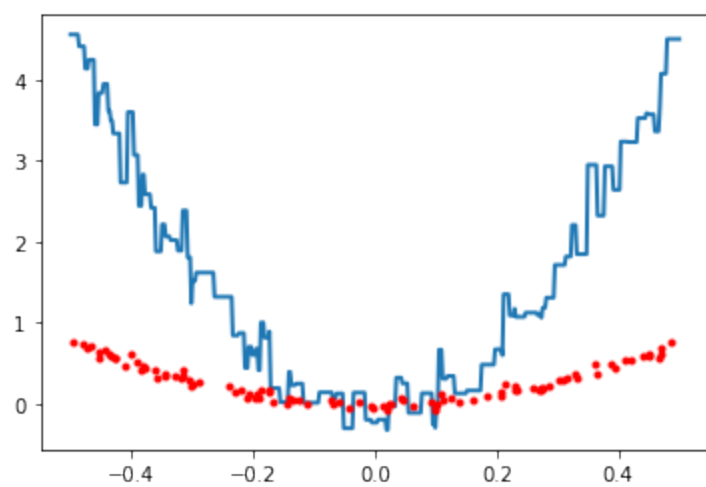
        regs.append(tree_reg)

        x1 = np.linspace(-0.5, 0.5, 500)
        y_pred = sum(lr * regressor.predict(x1.reshape(-1, 1)) for regressor in regs)

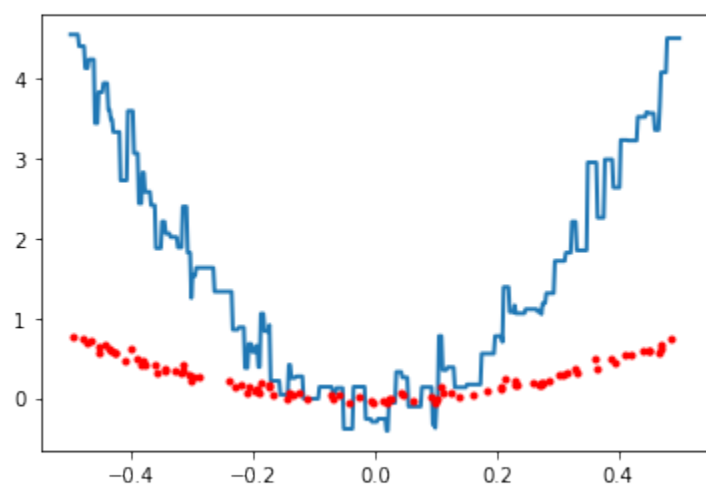
        print(number)
        plt.figure()
        plt.plot(x1, y_pred, linewidth=2)
        plt.plot(X[:, 0], foo, "r.")
        plt.show()

        gradient_boost(X,y,number-1,lr,count+1,regs,foo=foo)
```

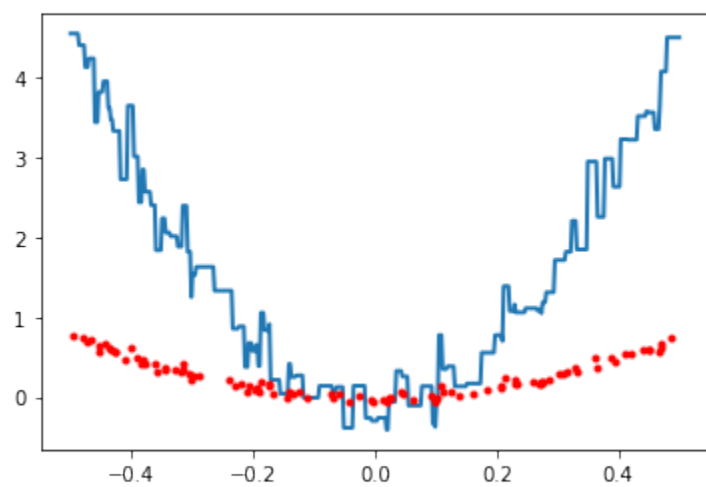
```
In [66]: np.random.seed(42)
X = np.random.rand(100, 1) - 0.5
y = 3*X[:, 0]**2 + 0.05 * np.random.randn(100)
gradient_boost(X,y,5,lr=1)
```



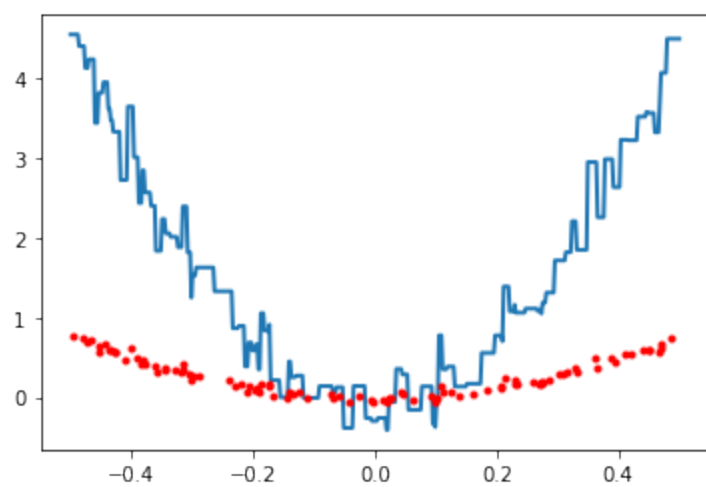
4



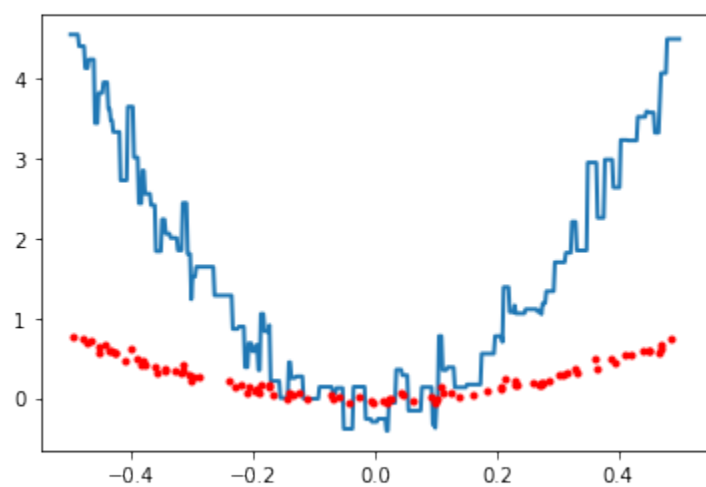
3



2



1



In []: